

กรณีศึกษาที่ 3: ระบบคำนวณยอดขายรวม

ร้านค้าออนไลน์จัดเก็บยอดขายรายวันไว้ในอาร์เรย์

```
double[] sales = {  
    1250.50, 890.00, 1575.25,  
    2300.00, 940.75  
};
```

ระบบต้องคำนวณยอดขายรวม ยอดขายเฉลี่ย และยอดขายสูงสุด

งานที่มอบหมาย

ออกแบบอัลกอริทึมที่อ่านข้อมูลในอาร์เรย์เพียงรอบเดียว

และคืนผลลัพธ์ทั้งสามค่า

สามารถสร้างคลาสสำหรับเก็บผลลัพธ์ เช่น

```
class SalesSummary {  
    double total;  
    double average;  
    double maximum;  
}
```

1. ข้อมูลนำเข้าและผลลัพธ์ที่ต้องการ

ข้อมูลนำเข้า (Input): อาร์เรย์ของตัวเลขทศนิยม double[] sales ที่เก็บยอดขายรายวัน

ผลลัพธ์ที่ต้องการ (Output): วัตถุ SalesSummary ที่บรรจุ 3 ค่า

ได้แก่:

- 1.) ยอดขายรวม (total)
- 2.) ยอดขายเฉลี่ย (average)
- 3.) ยอดขายสูงสุด (maximum)

2. ความหมายของขนาดข้อมูล (n)

n คือ จำนวนสมาชิก (ขนาดของอาร์เรย์) ใน sales จำนวนวันทั้งหมดที่มีการบันทึกยอดขาย

3. แนวคิด หรือ Pseudocode

Algorithm calculateSalesSummary(sales, n):

if n == 0:

 return null

total = 0.0

maximum = sales[0]

for i = 0 to n - 1 do:

 total = total + sales[i]

 if sales[i] > maximum then:

 maximum = sales[i]

 end if

end for

average = total / n

return SalesSummary(total, average, maximum)

4. โปรแกรมภาษา Java

```
class SalesSummary {
    double total;
    double average;
    double maximum;

    public SalesSummary(double total, double average, double maximum) {
        this.total = total;
        this.average = average;
        this.maximum = maximum;
    }
}

public class Main {
    public static SalesSummary calculateSalesSummary(double[] sales) {
        if (sales == null || sales.length == 0) {
            return new SalesSummary(0, 0, 0);
        }

        double total = 0;
        double maximum = sales[0];

        // วนลูปอ่านข้อมูลเพียงรอบเดียว
        for (int i = 0; i < sales.length; i++) {
            total += sales[i];
            if (sales[i] > maximum) {
                maximum = sales[i];
            }

            double average = total / sales.length;

            return new SalesSummary(total, average, maximum);
        }

        public static void main(String[] args) {
            double[] sales = {1250.50, 890.00, 1575.25, 2300.00, 940.75};
            SalesSummary summary = calculateSalesSummary(sales);

            System.out.println("Total: " + summary.total);
            System.out.println("Average: " + summary.average);
            System.out.println("Maximum: " + summary.maximum);
        }
    }
}
```

5. คำสั่งหลัก หรือ Basic Operation ที่ใช้วิเคราะห์

ตอบ: Basic Operation: คำสั่งการบวกเพื่อสะสมยอดรวม ($total += sales[i]$) และการเปรียบเทียบค่าเพื่อหาค่าสูงสุด ($sales[i] > maximum$) ภายในลูป

6. วิเคราะห์จำนวนครั้งที่คำสั่งหลักทำงาน

ตอบ: ในการวนลูป 1 รอบ จะมีการบวกและตรวจสอบเงื่อนไขอย่างละ 1 ครั้ง สำหรับอาร์เรย์ขนาด n ลูปจะทำงานทั้งหมด n ครั้ง

7. วิเคราะห์กรณี Best Case, Worst Case และ Average Case

ตอบ: Best Case: ต้องเข้าถึงและอ่านข้อมูลครบทุกตัวในอาร์เรย์ ทำงาน n ครั้ง

Worst Case: ต้องอ่านข้อมูลครบทุกตัวในอาร์เรย์ ทำงาน n ครั้ง

Average Case: ต้องอ่านข้อมูลครบทุกตัวเช่นกัน ทำงาน n ครั้ง

8. ระบุ Time Complexity ในรูป Big O, Omega, Theta

ตอบ: Big O (Worst Case): $O(n)$

ในกรณีแย่ที่สุด ลูป for จะต้องอ่านข้อมูลสมาชิกทุกตัวใน Array sales ตั้งแต่ดรรชนี 0 ถึง $n-1$ รวม n รอบ

Big Omega (Best Case): $\Omega(n)$ (หรือ $\Omega(1)$) หากพิจารณากรณีพิเศษ $sales == null$ หรือขนาด 0)

เมื่อมีข้อมูลใน Array ($n > 0$) อัลกอริทึมไม่มีเงื่อนไขเลิกทำก่อน (Early Exit) จึงต้องวนทำงานครบ n รอบเสมอ

Big Theta (Tight Bound): $\Theta(n)$

เนื่องจากขอบเขตบน (O) และขอบเขตล่าง (Ω) เติบโตในอัตราเชิงเส้นเท่ากัน อัลกอริทึมนี้จึงมีค่า Tight Bound เป็น $\Theta(n)$

9. วิเคราะห์ Auxiliary Space Complexity

ตอบ: $O(1)$ (Constant Space) เพราะ Auxiliary Space คิดเฉพาะหน่วยความจำชั่วคราวที่ถูกสร้างเพิ่มในฟังก์ชัน โดยไม่นับ Input Array (sales)

ในฟังก์ชันนี้มีการสร้างเพียงตัวแปรชนิด primitive ไม่กี่ตัว ได้แก่ total, maximum, average และตัวนับ i ซึ่งใช้พื้นที่หน่วยความจำคงที่ ไม่ได้แปรผันตามขนาด n ของ Array

10. เสนอแนวทางปรับปรุงอัลกอริทึมให้มีประสิทธิภาพมากขึ้น

ตอบ: แม้ว่าอัลกอริทึมนี้จะอยู่ในระดับ $O(n)$ ซึ่งเป็นระดับที่เหมาะสมอยู่แล้วในการหาค่า Sum/Max แต่สามารถปรับปรุงในระดับ Implementation และ Architecture ได้ดังนี้:

1. ใช้ Parallel Processing / Java Streams สำหรับข้อมูลขนาดใหญ่ (Large Scale Data):

หาก Array มีขนาดใหญ่มาก (เช่น ล้านรายการ) สามารถประมวลผลแบบขนานด้วย `Arrays.stream(sales).parallel()` หรือแบ่ง Workload ด้วย Fork/Join Framework เพื่อกระจายงานไปให้หลาย CPU Cores

2. ปรับใช้ Loop Unrolling (ลด Overhead ของการวนลูป):

ประมวลผลข้อมูลครั้งละ 2-4 ตัวใน 1 รอบของลูป เพื่อลดจำนวนครั้งในการเช็คเงื่อนไข $i < sales.length$ และการเพิ่มค่า $i++$

3. การปรับปรุงเชิงโครงสร้าง (Maintain State):

หากข้อมูล sales ถูกบันทึกหรืออัปเดตผ่าน Data Structure เฉพาะ (เช่น Custom Class) อาจคำนวณ total และค่า maximum สะสมไว้ตั้งแต่ตอนที่มีการเพิ่มข้อมูลเข้าระบบ (Insert Time) เพื่อให้เวลาเรียกใช้อ่านค่าสรุปทำได้ทันทีด้วยเวลา $O(1)$

คำถามวิเคราะห์

1. คำสั่งหลักใดทำงาน (n) ครั้ง

ตอบ: คำสั่งในลูป for:

1. `total += sales[i];`
2. `if (sales[i] > maximum)` (การเปรียบเทียบค่า)
3. `i++` และการเช็คเงื่อนไข `i < sales.length`

เมื่อ Array `sales` มีขนาด `n` คำสั่งที่อยู่ภายในบล็อกลูป `for` จะถูกเรียกประมวลผลทั้งหมด `n` รอบพอดี

2. Time Complexity ของอัลกอริทึมเป็นเท่าใด

ตอบ: $O(n)$ (Linear Time Complexity) เพราะฟังก์ชันมีการทำงานหลักผ่านลูป `for` เพียงลูปเดียวที่วนอ่านข้อมูลตั้งแต่ดัชนี 0 ถึง `n-1` การเติบโตของเวลาประมวลผลจึงเป็นสัดส่วนตรงกับขนาดข้อมูล `n`

3. Auxiliary Space Complexity เป็นเท่าใด

ตอบ: $O(1)$ (Constant Space Complexity)

เพราะ Auxiliary Space คิดเฉพาะหน่วยความจำชั่วคราวที่สร้างขึ้นในฟังก์ชัน

โค้ดนี้สร้างเพียงตัวแปรชนิด Primitive ไม่กี่ตัว ได้แก่ `total`, `maximum`, `average` และ `i` ซึ่งใช้พื้นที่คงที่เสมอ โดยไม่ได้สร้าง Array หรือ Data Structure ใหม่ตามขนาด `n`

4. Best Case และ Worst Case แตกต่างกันหรือไม่

ตอบ: ไม่แตกต่างกัน (ทั้ง Best Case และ Worst Case เป็น $O(n)$ เท่ากัน)

เพราะอัลกอริทึมไม่มีคำสั่งหยุดลูปก่อนกำหนด (เช่น `break` หรือ `return` กลางลูป)

ไม่ว่าข้อมูลใน Array จะเรียงแบบใด (มากไปน้อย น้อยไปมาก หรือสุม) ลูปก็ยังคงต้องวนทำงานจนครบ `n` รอบเท่าเดิมเสมอ

5. หากแยกเขียนลูป 3 รอบ

ความซับซ้อนจะเปลี่ยนจากวิธีลูปรอบเดียวหรือไม่

ตอบ: ไม่เปลี่ยนแปลง (ยังคงเป็น $O(n)$ เหมือนเดิม)

เพราะ ลูปที่ 1 หา `Total`, ลูปที่ 2 หา `Max`, ลูปที่ 3 หา `Average` หรือประมวลผลอื่น

เวลาทำงานรวมจะเป็น $n + n + n = 3n$

ตามหลักการคำนวณ Big O Notation เราจะตัดค่าคงที่ (Constant Coefficient) ออก ดังนั้น $O(3n) \rightarrow O(n)$ ทำให้ความซับซ้อนเชิงทฤษฎียังคงเดิม

6. แม้ Big O จะเท่ากัน เหตุใดวิธีลูปรอบเดียวจึงอาจทำงานเร็วกว่า

ตอบ: เกิดจากเหตุผลทางด้าน Low-level Hardware และ Runtime ดังนี้:

1. ลด **Loop Overhead**: การเขียน 1 ลูป จะมีการเช็คเงื่อนไข `i < sales.length` และการเพิ่มค่า `i++` เพียง `n` ครั้ง แต่ถ้าใช้ 3 ลูป จะต้องทำถึง `3n` ครั้ง
2. ประโยชน์จาก **CPU Cache (Locality of Reference)**: การเข้าถึง Array `sales[i]` ครบทุกการทำงานในลูปเดียว ช่วยให้ CPU อ่านข้อมูลจาก **Cache Memory** ได้อย่างมีประสิทธิภาพสูงสุด ต่างจากการวนลูป 3 รอบที่ CPU ต้องโหลดข้อมูลใน Array จาก Main Memory เข้า Cache ใหม่หลายครั้ง